

Introduction

This documentation describes a complete end-to-end Retrieval-Augmented Generation (RAG) chatbot system built using n8n, PostgreSQL, Qdrant vector database, and OpenAI. The system enables intelligent question-answering based on a curated knowledge base of blog articles from Bitovi's website.

Overview

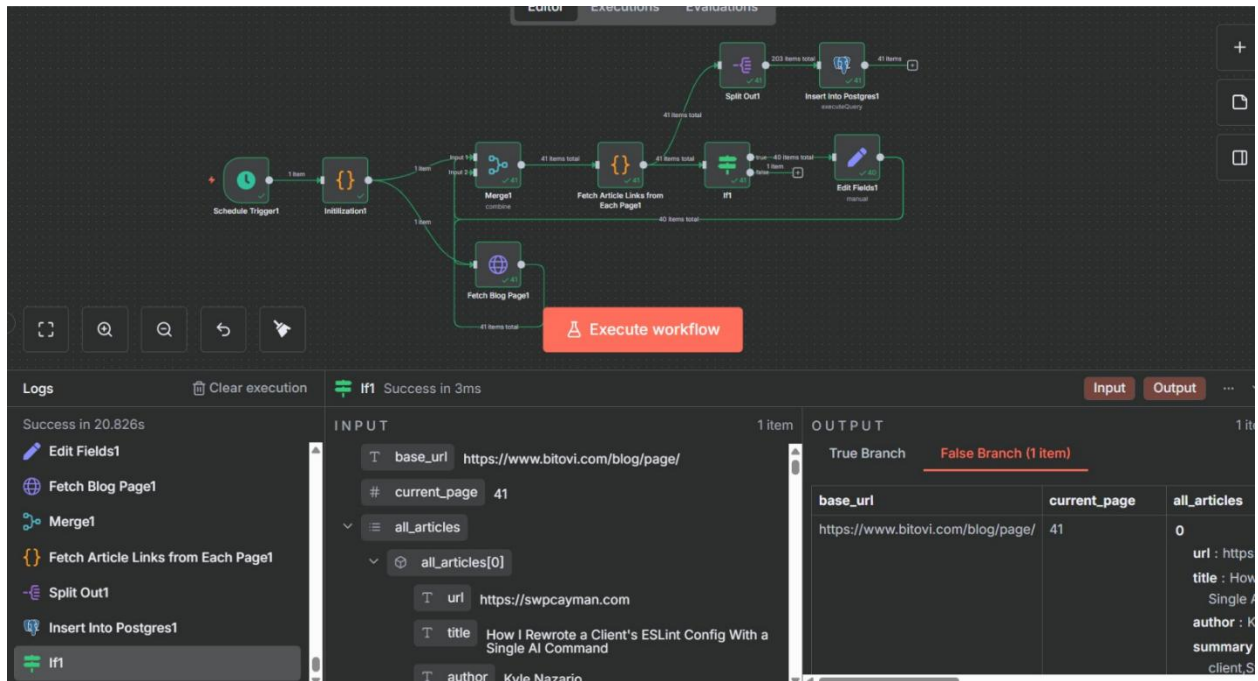
The RAG chatbot system consists of 3 main workflows that work together to create a knowledge-based conversational AI:

1. **Web Scraping & Storage Pipeline:** Automatically detects and scrapes blog articles from the Bitovi website, extracting structured metadata (title, author, topic, summary, URL) and storing it in a PostgreSQL database with built in duplicate prevention.
2. **Vector Embedding Pipeline:** Processes articles from PostgreSQL, generates semantic embeddings using OpenAI's embedding model, and stores them in a Qdrant vector database for efficient similarity search. Includes duplicate detection to prevent re-embedding existing articles.
3. **RAG Query & Response System:** A dual-mode chatbot that handles user queries through two distinct pathways:
 - **Semantic Search Mode:** For content based questions, it performs vector similarity search to retrieve relevant articles and generates contextual answers using GPT-4o-mini
 - **Metadata Query Mode:** For filtering and counting queries (e.g., "how many React blogs?"), it performs topic based filtering and returns structured lists

Tech Stack

- **Workflow Orchestration:** n8n
- **Relational Database:** PostgreSQL
- **Vector Database:** Qdrant
- **AI Models:** OpenAI (text-embedding-3-small for embeddings, GPT-4o-Mini for generation)
- **Programming:** Python (scraping), JavaScript (data transformation)

Step 1: Web Scraping & Storage Pipeline



Automatically scrapes blog articles from Bitovi's website and stores them in PostgreSQL.

Nodes:

1. **Schedule Trigger** - Runs the workflow automatically (At midnight)
2. **Initialization** - Sets starting page to 1 and base URL
3. **Merge** - Combines page data with HTML content
4. **Fetch Blog Page** - Downloads the HTML from the current page URL
5. **Fetch Article Links from Each Page** - Extracts article data (title, author, topic, summary, URL) from HTML using Python
6. **Split Out** - Splits the articles array into individual items
7. **Insert Into Postgres** - Inserts each article into the database
8. **Merge All Data** - Restores page level data structure
9. **Edit Fields** - Increments page number (page 1 → page 2 → page 3...)
10. **IF** - Checks if more pages exist
 - **TRUE**: Loops back to fetch next page
 - **FALSE**: Workflow ends

PostgreSQL » pg-rdb » rdb » public » SQL command

SQL command

```
SELECT id, url, title, author, topic, summary
FROM bitovi_articles
LIMIT 5
```

id	url	title	author	topic	summary
1	https://wpcayman.com	How I Rewrote a Client's ESLint Config With a Single AI Command	Kyle Nazario		I recently refactored some code for a Bitovi client, Strategic Wealth Preservation. We used GitHub Co
2	https://www.bitovi.com/blog/the-complete-guide-to-devops	Interview with a Bitovian: Reed Phil Hemming, DevOps Engineer	The Bitovi Team	DevOps	Phil is a Site Reliability Engineer who works with clients to identify their DevOps and infrastructure ne
3	https://www.bitovi.com/services/frontend/angular-consulting	Boost Domain-Driven Architecture with Rx, Angular, and NestJS	Dynas El-Rouh	Angular	Powerful and scalable apps require powerful and scalable tech stacks. Combining Angular, NestJS, an
4	https://github.com/bitovi/smartcomponents	Introducing AI Component Faste: An Open Source AI Form Filler	David Nicholas	React	With AI booming, we're always on the lookout for ways it can deliver maximum value. That's why a
5	https://www.invisionapp.com/inside-design/a-beginners-guide-to-querrita-research/	Fast Food Usability Series: Wendy's Case Study	Marc Madjaric	Design	This is part one in a multi-part series on usability issues with fast food eCommerce websites. We ve

5 rows (show 5) Edit, Explain, Export

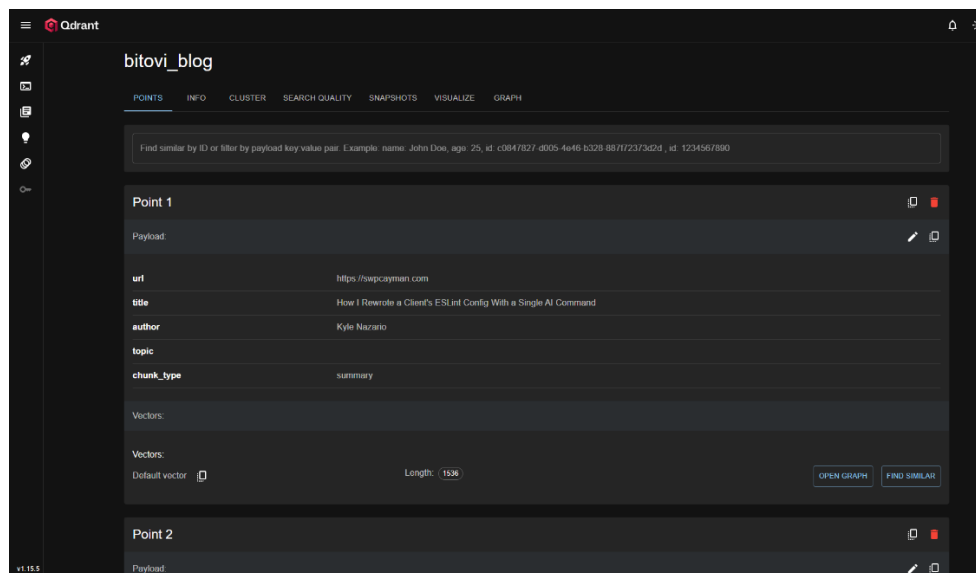
```
SELECT id, url, title, author, topic, summary
FROM bitovi_articles
LIMIT 5
```

Step 2: Vector Embedding Pipeline

Takes articles from PostgreSQL, creates vector embeddings, and stores them in Qdrant for semantic search.

Nodes:

1. **Schedule Trigger** - Runs automatically to check for new articles
2. **Execute a SQL query** - Fetches articles from PostgreSQL that need embedding
3. **Check if URL exists in Qdrant** - Queries Qdrant to see if article is already embedded
4. **IF** - Checks if the article is new
 - **TRUE**: Article doesn't exist in Qdrant, continue to embed
 - **FALSE**: Skip (article already embedded)
5. **Create Embeddings** - Sends article content to OpenAI to generate vector embedding
6. **Merge** - Combines the embedding with article metadata
7. **Collect Data** - Formats data for Qdrant (ID, vector, payload)
8. **Upsert Points** - Stores the vector and metadata in Qdrant



Step 3: RAG Query & Response System

Answers user questions using two modes: semantic search for content questions and metadata filtering for counting/listing questions.

Initial Flow:

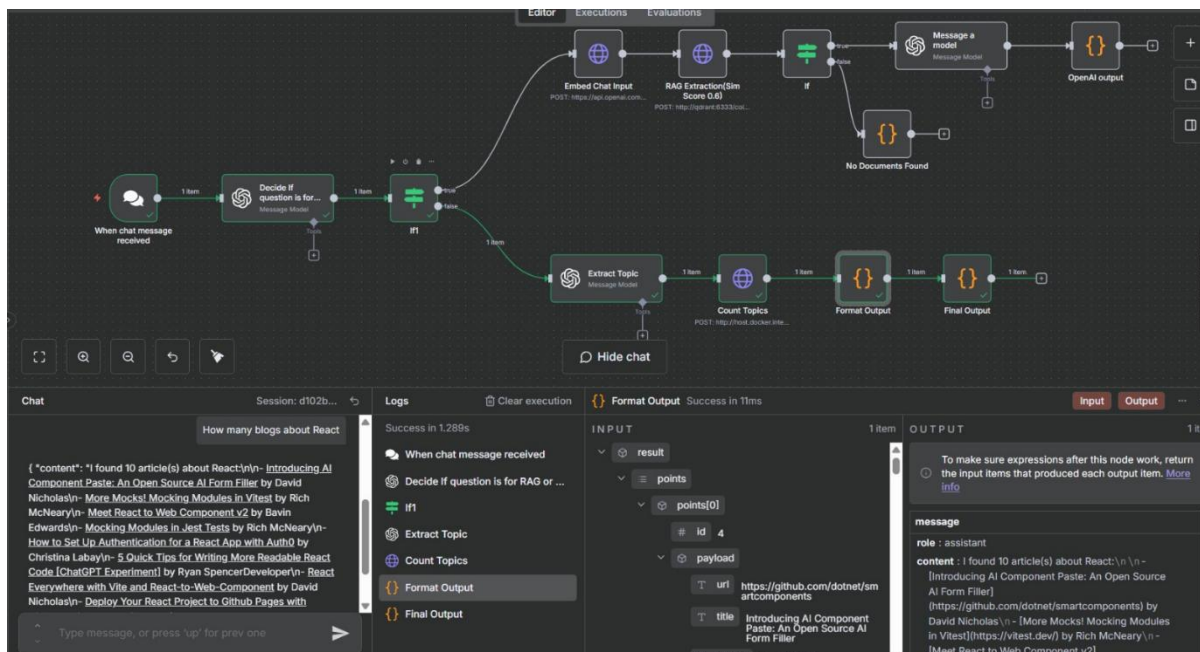
1. **When chat message received** - User types a question
2. **Decide if question is for RAG or NOT** - AI classifies the question type
3. **IF** - Routes to appropriate pathway

Path 1: Metadata Query (Counting/Filtering)

Used for: "How many React blogs?", "List all Angular articles"

Nodes:

1. **Extract Topic** - AI extracts the topic name from the question (e.g., "React")
2. **Count Topics** - Queries Qdrant filtering by topic
3. **Format Output** - Counts results and formats response with article list
4. **Final Output** - Displays clean text in chat



Path 2: Semantic Search (RAG)

Used for: "What are React best practices?", "How to use React Suspense?"

Nodes:

1. **Embed Chat Input** - Converts question to vector embedding
2. **RAG Extraction** - Searches Qdrant for similar articles (similarity score > 0.3)
3. **IF** - Checks if results were found
 - **TRUE:** Send to OpenAI with context
 - **FALSE:** Show "No related blogs found"
4. **Message a model** - GPT-4o-mini answers based on retrieved articles
5. **OpenAI output** - Extracts and displays response in chat

Returns: AI generated answer with citations and article links based only on the provided context.

